

任务

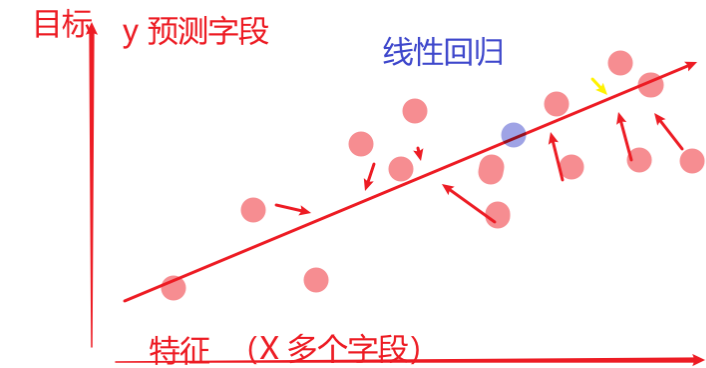
任务：根据历史数据，建立模型，然后根据新的特征，预测目标

mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
1	8	250	165	3504	11.5	70	1	chevrolet chevell
2	8	318	150	3436	11	70	1	buick skylark 320
3	8	302	140	3449	10.5	70	1	plymouth satellit
4	8	429	198	4341	10	70	1	amc rebel sst
5	8	454	220	4354	9	70	1	ford torino
6	8	440	215	4312	8.5	70	1	ford galaxie 500
7	8	455	225		10	70	1	chevrolet impala
8	8	390	190	3850	8.5	70	1	plymouth fury iii
9	8	383	170a	3563	10	70	1	pontiac catalina
10	8	340	160	3609	8	70	1	amc ambassador dp
11	8	400	150	3761	9.5	70	1	dodge challenger
12	8						1	plymouth 'cuda 34
13	8						1	chevrolet monta c
14	8						1	
15	8						1	

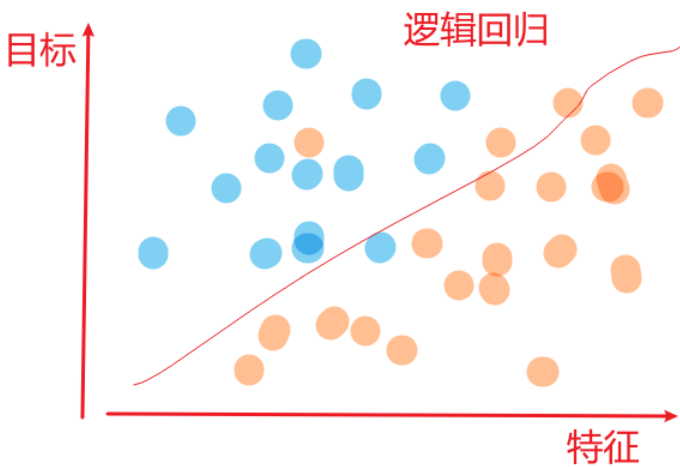
使用方法

机器学习

线性回归



逻辑回归



机器学习的大致流程

- 机器学习是有门槛
 - 1. 所有机器学习的参与的数据都是要数值的（不满足报错），使用数据清洗
 - 2. 机器学习的数据不能太离散（不满足，无法取得优秀的结果）使用数据标准化防范
- 机器学习的大致流程
 - 输入导入
 - 空值，异常值处理
 - 数据类型转换
 - 数据标准化
 - 数据的划分

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
y_train	16	8	350	130	3504	12	70	1	chevrolet chevelle
	15	8	350	165	3693	11.5	70	1	buick skylark 320
	18	8	318	150	3436	11	70	1	plymouth satellite
	17	8	304	150	3436	11	70	1	amc rebel sst
	17	8	302	140	3449	10.5	70	1	ford torino
	15	8	429	198	4341	10	70	1	ford galaxie 500
	14	8	454	220	4354	9	70	1	chevrolet impala
	14	8	440	215	4312	8.5	70	1	plymouth fury iii
	14	8	455	225	4312	10	70	1	pontiac catalina
y_test	15	8	390	190	3850	8.5	70	1	amc ambassador dpl
	15	8	383	170a	3563	10	70	1	dodge challenger se
	14	8	340	160	3609	9.5	70	1	plymouth cuda 340
	15	8	400	150	3761	9.5	70	1	chevrolet monte car

- 机器学习
- 机器学习各一个评分结果
- 保存模型下次使用

● 机器学习的上半场2.1.X（机器学习的预处理阶段）

● 2.1.1

```

# 检查缺失值并删除缺失值所在的行 2分
print("\n检查缺失值:")
print(data.isnull().sum())
data = data.dropna()

# 将 'horsepower' 列转换为数值类型，并（删除）处理转换中的异常值 1分
data['horsepower'] = (data['horsepower'], errors='coerce')
data = 

# 显示每一列的数据类型

```

[3]:

```

import pandas as pd
pd.to_*?

pd.to_datetime
pd.to_numeric
pd.to_pickle
pd.to_timedelta

```

```

from sklearn.preprocessing import StandardScaler
# 对数值型数据进行标准化处理 1分
numerical_features = ['displacement', 'horsepower', 'weight', 'acceleration']
scaler = StandardScaler()
data[numerical_features] = scaler.fit_transform(data[numerical_features])

from sklearn.model_selection import train_test_split
# 选择特征、自变量和目标变量 2分
selected_features = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model year', 'origin', 'car name']
X = data[selected_features]
y = data['mpg']

```

大写的类
小写的是方法
函数
类就是一个模板
通过实例化后，就可以继承这些模板

标注化的基本流程

```

from sklearn.preprocessing import StandardScaler
# 对数值型数据进行标准化处理 1分
numerical_features = ['displacement', 'horsepower', 'weight', 'acceleration']
scaler = StandardScaler()
data[numerical_features] = scaler.fit_transform(data[numerical_features])

from sklearn.model_selection import train_test_split
# 选择特征、自变量和目标变量 2分
selected_features = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model year', 'origin', 'car name']
X = data[selected_features]
y = data['mpg']

```

1 导入类
2 实例化，学习类的所有本领
3
4
5 将等号左边的放入括号

```

data.columns
[5]:
Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
      'acceleration', 'model year', 'origin', 'car name'],
      dtype='object')

```

返回表的字段列表，避免手写错误

```

from sklearn.model_selection import train_test_split
# 选择特征、自变量和目标变量 2分
selected_features = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model year', 'origin', 'car name']
X = data[selected_features]
y = data['mpg']

```

文本字段未处理
不能直接机器学习

```

data[numerical_features] = scaler.fit_transform(data[numerical_features])

from sklearn.model_selection import train_test_split
# 选择特征、自变量和目标变量 2分
selected_features = ['cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model year', 'origin']
X = data[selected_features]
y = data['mpg']

```

data 不要忘记

```

from sklearn.model_selection import train_test_split
# 选择特征、自变量和目标变量 2分
selected_features = ['cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model year', 'origin']
X = data[selected_features]
y = data['mpg']

# 划分数据集为训练集和测试集（训练集占8成） 1分
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

2.1.2

```
[ ]:
import pandas as pd
#读取一个Excel文件，并将读取到的数据存储在变量data中
data = pd.read_excel('大学生低碳生活行为的影响因素数据集.xlsx')
#打印出数据集的前5行
print(data.head())
```

```
#打印出数据集的前5行
print(data.head())

#处理数据集中的缺失值
initial_row_count = data.shape[0] #处理前的数据行数
data = data.dropna() #删除缺失值所在行
final_row_count = data.shape[0] #处理后的数据行数
print(f'处理后数据行数: {final_row_count}, 删除的行数: {initial_row_count - final_row_count}')

#删除重复行
```

等同于len() 函数
这个是属性，没有括号
取第一个元素就是行数量
第二个元素是列数量

Index with the appropriate dtype instead.

```
ns[key] = getattr(obj, key)
pd.Series.drop
pd.Series.drop_duplicates
pd.Series.droplevel
pd.Series.dropna
```

```
[1]:
import pandas as pd
pd.Series.dro*?
```

```
[ ]:
```

```
from sklearn.preprocessing import StandardScaler 1
numerical_features = ['4.您的月生活费o≤1,000元 o1,001-2,000元 o2,001-3,000元 o≥3,000元'] 2
scaler = StandardScaler()
data[numerical_features] = scaler.fit_transform(data[numerical_features]) 3 4 5
#选择特征
```

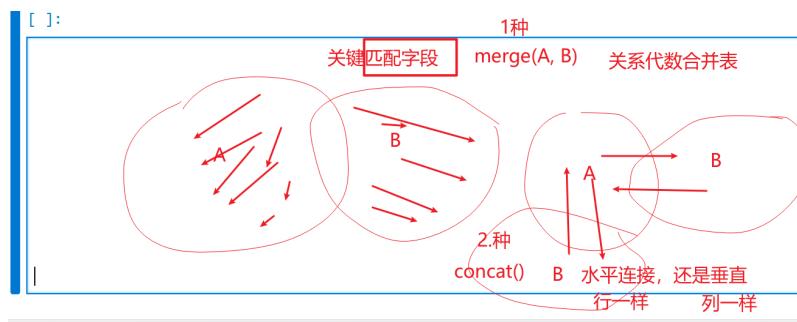
```
[4]:
data.columns[2:10]

Index(['1.您的性别o男性 o女性', '2.您的年级o大一 o大二 o大三 o大四',
       '3.您的生源地o农村 o城镇(乡镇) o地县级城市 o省会城市及直辖市',
       '4.您的月生活费o≤1,000元 o1,001-2,000元 o2,001-3,000元 o≥3,001元',
       '5.您进行过绿色低碳的相关生活方式吗?', '6.您觉得“低碳”，与你的生活关系密切吗?', '7.低碳生活是否会成为未来的主流生活方式?',
       '8.您是否认为低碳生活会提高您的生活质量?'],
      dtype='object')
```

从第三个开始，连续取8个

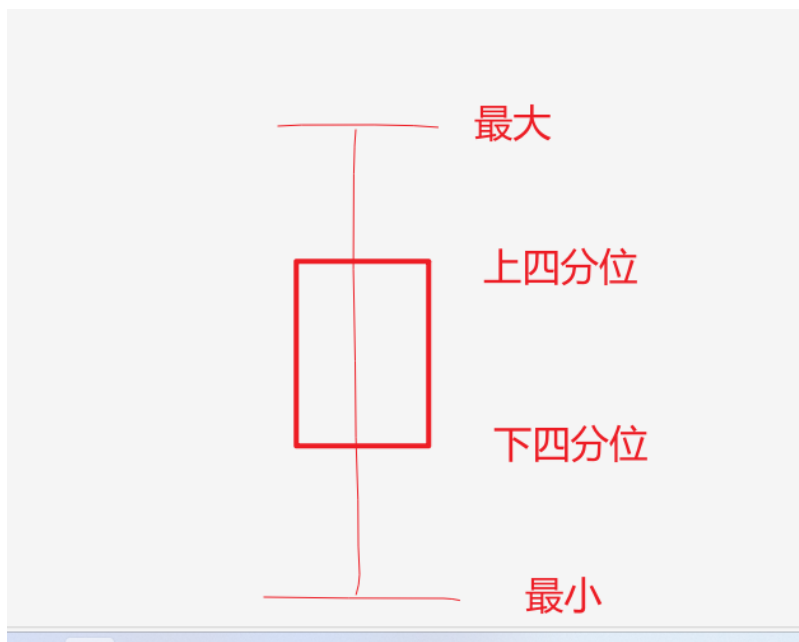
合并表

- 通过关键字段 merge()
- 通过索引concat()
- 可以通过水平方向直接合并，也可通过垂直方向直接合并



2.1.3

- 箱型图



步骤1: 数据排序

[43, 59, 62, 69, 72, 75, 77, 79, 83, 85, 88, 89, 91, 94, 105]

步骤2: 计算四分位数

- Q1 (第25百分位数) : 第4个数据 (位置=15×25%=3.75→向上取整为4) → 69
- Q2 (中位数) : 第8个数据 (位置=15×50%=7.5→向上取整为8) → 79
- Q3 (第75百分位数) : 第12个数据 (位置=15×75%=11.25→向上取整为12) → 89

步骤3: 计算IQR和异常值边界

- IQR = Q3 - Q1 = 89 - 69 = 20
- 下界 = Q1 - 1.5×IQR = 69 - 1.5×20 = 39
- 上界 = Q3 + 1.5×IQR = 89 + 1.5×20 = 119

IQR 计算异常值

```
# 定义特征和目标
X = data_cleaned.drop(columns=[target_variable, 'Unnamed: 0']) #1分
y = data_cleaned[target_variable] #1分

# 划分数据 (训练集占80%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 显示划分后的数据形状
```

根据pdf中特征数量, 这个也要删除

2.1.4

```
# 修改列名
data.rename(columns={"病人ID": "患者ID"}, inplace=True)

# 查看修改后的数据
print(data)

# 增加列
data['诊断'] = data['患者ID'] + data['症状']
```

shift + Tab 键看提示

```
from datetime import datetime

# 增加诊断延迟和病程列
data['诊断延迟'] = (data['诊断日期'] - data['就诊日期']).dt.days
data['病程'] = (datetime(2024, 9, 1) - data['诊断日期']).dt.days

# 删除不合理的数据
data = data[(data['诊断延迟'] >= 0) & (data['病程'] > 0) & (data['年龄'] < 120)]

# 查看修改后的数据
```

写反就报错

```
# 增加诊断延迟和病程列
data['诊断延迟'] = (data['诊断日期'] - data['就诊日期']).dt.days
data['病程'] = (datetime(2024, 9, 1) - data['诊断日期']).dt.days

# 删除不合理的数据
data = data[(data['诊断延迟'] >= 0) & (data['年龄'] > 0) & (data['年龄'] < 120)]

# 查看修改后的数据
```

```
# 统计治疗结果分布
treatment_outcome_distribution = data.groupby('疾病类型')['治疗结果'].value_counts().unstack()

# 设置中文字体
font_path = 'C:/Windows/Fonts/simhei.ttf' # 根据你的系统调整字体路径
my_font = fm.FontProperties(fname=font_path)

# 绘制柱状图
treatment_outcome_distribution.plot(kind='bar', stacked=True)
plt.title('不同疾病类型的治疗结果分布', fontproperties=my_font)
plt.xlabel('疾病类型', fontproperties=my_font)
plt.ylabel('治疗结果数量', fontproperties=my_font)
```

只要导入了matplotlib

pandas 就可以画出简单的图标，后续复杂

设置，由matplotlib 继续完成

```
font_path = 'C:/Windows/Fonts/simhei.ttf' # 根据你的系统调整字体路径
my_font = fm.FontProperties(fname=font_path)
```

shift + tab 查看帮助

```
# 绘制柱状图
treatment_outcome_distribution.plot(kind='bar', stacked=True)
plt.title('不同疾病类型的治疗结果分布', fontproperties=my_font)
plt.xlabel('疾病类型', fontproperties=my_font)
plt.ylabel('治疗结果数量', fontproperties=my_font)
plt.xticks(fontproperties=my_font)
plt.yticks(fontproperties=my_font)
plt.legend(prop=my_font) # 设置图例字体
plt.show()
```

The kind of plot to produce:

- 'line': line plot (default)
- 'bar': vertical bar plot
- 'barh': horizontal bar plot
- 'hist': histogram

```
# 绘制柱状图
treatment_outcome_distribution.plot(kind='bar', stacked=True)
plt.title('不同疾病类型的治疗结果分布', fontproperties=my_font)
plt.xlabel('疾病类型', fontproperties=my_font)
plt.ylabel('治疗结果数量', fontproperties=my_font)
plt.xticks(fontproperties=my_font) # 设置x轴刻度标签的字体
plt.yticks(fontproperties=my_font) # 设置y轴刻度标签的字体
plt.legend(prop=my_font) # 设置图例字体
plt.show()
```

图标题

图例

销量

1000

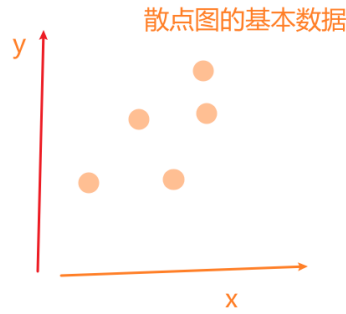
1月

月份

```
# 绘制散点图
plt.figure(figsize=(10, 5))
plt.title('年龄和疾病严重程度度的关系', fontproperties=my_font)
```

```
fontproperties=my_font)
量', fontproperties=my_font)
ties=my_font) # 设置x轴刻度标签的字体
ties=my_font) # 设置y轴刻度标签的字体
nt) # 设置图例字体

, _____)
重程度的关系', fontproperties=my_font)
ntproperties=my_font)
度', fontproperties=my_font)
ties=my_font) # 设置x轴刻度标签的字体
ties=my_font) # 设置y轴刻度标签的字体
```



2.1.5

```
# 填充缺失值
data_filled = data.apply(lambda x: x.fillna(x.mode()[0]))

# 划分数据 (测试集占比20%)
train_data, test_data = train_test_split(data_filled, test_size=0.2, random_state=42)

# 保存处理后的数据
cleaned_file_path = '_____'

exercise_frequency_counts = data_cleaned['How often do you exercise?'].value_counts()

# 绘制饼图
plt.figure(figsize=(10, 6))
exercise_frequency_counts.plot.pie(autopct='%1.1f%%', startangle=90, colors=plt.cm.Paired)
plt.title('Distribution of Exercise Frequency')
plt.ylabel('')
plt.show()

import pandas as pd
```

直接填入

这里关门了, 所以

pie 写到前面去

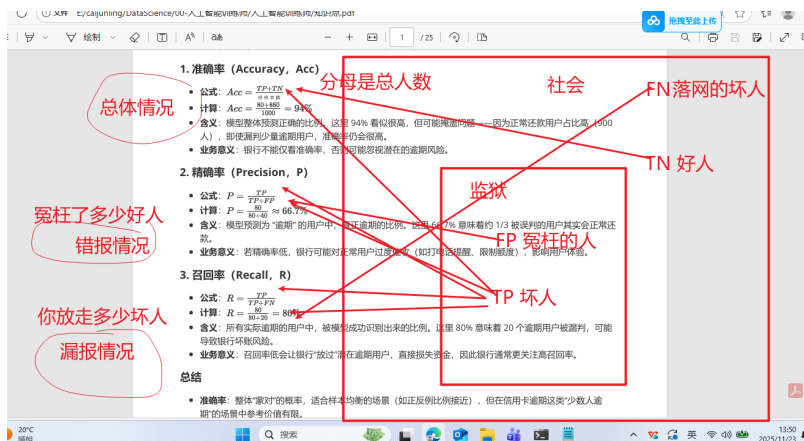
机器学习的下半场2.2.X (完整机器学习, 重点是评估模型)

2.2.1

```
[ ]:
import pandas as pd
from sklearn.model_selection import train_test_split 数据划分
from sklearn.linear_model import LogisticRegression 逻辑回归-处理二分类问题
import pickle 将模型记录到本地, 下次还可以使用
from sklearn.metrics import classification_report 评估模型
from imblearn.over_sampling import SMOTE 重采样

# 加载数据
data = _____

# 电子商务的数据
```

2.2.2

```
[ ]:
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import pickle
from sklearn.ensemble import RandomForestRegressor
```

加载数据集

线性回归，预测数值

将机器学习步骤打包，方便管理，增加代码复用

随机森林，捕捉一些非线性的

```
del df['horsepower'] = df['horsepower'].dropna()
df = df.dropna(subset=['horsepower'])

# 选择相关特征进行建模（定义自变量（返回一个DataFrame）和因变量）
X = df[['cylinders', 'displacement', 'horsepower', 'weight',
        'acceleration', 'model year', 'origin']]
y = df['mpg']

# 将数据集划分为训练集和测试集（测试集占比20%）
```

两个中括号，mpg

car_name 不要

```
# 将数据集划分为训练集和测试集（测试集占比20%）
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 创建包含标准化和线性回归的管道
pipeline = Pipeline([('scaler', StandardScaler()), ('linreg', LinearRegression())])

# 训练模型

# 保存训练好的模型
```

括号不要忘记

pipeline 将标准化和线性回归，一次全部加载到管道中

```
+ Code Python 3 (ipykernel)
```

```
results_file.write(f'训练集得分: {pipeline.score(X_train, y_train)}\n')
results_file.write(f'测试集得分: {pipeline.score(X_test, y_test)}\n')
```

shift + tab 键 查看帮助

```
# 创建随机森林回归模型实例（创建的决策树的数量为100）
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

# 训练随机森林回归模型

# 使用随机森林模型进行预测
y_pred_rf = rf_model.predict(X_test)

# 保存新的结果
results_rf_df = pd.DataFrame({'2.2.2_results_rf': y_pred_rf})
```

Init signature: RandomForestRegressor(n_estimators=100, *, criterion='squared_error', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0)

2.2.3

```
[2]:
import pandas as pd
pd.get*?

pd.get_dummies
pd.get_option
```

```
[ ]:
```

```
xgb.XGB*?

xgb.XGBClassifier
xgb.XGBModel
xgb.XGBRFClassifier
xgb.XGBRFRegressor
xgb.XGBRanker
xgb.XGBRegressor
```

通过通配符查找xgb的方法

```
[ ]:
```

```
report_file.write(f' Parameters
report_file.write(f'
report_file.write(f'
report_file.write(f'
n_estimators : int
Number of gradient boosted trees. Equivalent to number
of boosting
rounds.
# 运用工具分析算法中错误案例
# 初始化XGBoost回归模型
xgb_model = xgb.XGBRegressor(, random_state=42)
# 训练XGBoost回归模型
# 使用XGBoost回归模型在测试集上进行结果预测
y_pred_xgb =
```

shift + tab 看帮助

```
to_csv('2.2.3_results_xgb.txt', index=False)

3_report_xgb.txt', 'w') as xgb_report_file:
file.write(f'XGBoost训练得分: {results_df_xgb.score(X_train, y_train)}\n')
file.write(f'XGBoost测试得分: {results_df_xgb.score(X_test, y_test)}\n')
file.write(f'XGBoost均方误差(MSE): {mean_squared_error(y_test, y_pred_xgb)}\n')
file.write(f'XGBoost决定系数(R^2): {r2_score(y_test, y_pred_xgb)}\n')
```

原题多了一个括号

不删除要报错

2.2.4

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import joblib
from xgboost import XGBRegressor
```

pickle 的平替, 保存模型

```
from sklearn.metrics import mean_squared_error, r2_score
import joblib
from xgboost import XGBRegressor

# 加载数据集
data = pd.read_excel('大学生低碳生活行为的影响因素数据集.xlsx')

# 显示数据集的前五行
print(data.head())

# 删除不必要的列并处理分类变量
data_cleaned = data.drop(columns=['序号', '所用时间']) # 删除不必要的列
```

```

Edit View Run kernel Settings Help
+ ✂ 📄 📄 ▶ ■ ↺ ▶▶ Code ▾ JupyterLab Python
# 将测试结果保存到报告文件
report_filename = report_filename
n_estimators : int
with open(report_filename, 'w') as f:
    Number of gradient boosted trees. Equivalent
    f.write(f'均方误差: {mse}\n')
    boosting
    f.write(f'平均绝对误差: {mae}\n')
    rounds.

# 分析并纠正错误 (显示)
max_depth : Optional[int]
# 初始化XGBoost模型
Maximum tree depth for base learners.
xgb_model = XGBRegressor(subsample=0.8, colsample_bytree=0.8)
# 训练XGBoost模型

shit + tab 看帮助

```

2.2.5

```

# 加载数据集
df = pd.read_csv("fitness_analysis.csv")
# 显示前五行数据
print(df.head())
# 选择相关特征进行建模
X = df[['Your gender', 'How important is exercise to you?', 'How healthy do you consider yourself?']]

y_pred = dt_model.predict(X_test)

# 将结果保存到文本文件中
results = pd.DataFrame({'实际值': y_test, '预测值': y_pred})
results_filename = '2.2.5_results.txt'
results.to_csv(results_filename, index=False, sep='\t')

# 将测试结果保存到报告文件中
report_filename = '2.2.5_report.txt'
with open(report_filename, 'w') as f:
    f.write(f'均方误差: {mse}\n')
    f.write(f'平均绝对误差: {mae}\n')
    f.write(f'决定系数: {r2}\n')

```

注意数据集叫df，不是data

参数不要忘记，'w' 代表写入文件

数据类型转换

astype() 强制类型转换，不能转就报错

- 1+1=11 (文本类型)
- 1+1+2 (数值类型)
- 小数转换成整数
- 无法转换就报错

pd.to_numeric() 转换中可以将错误数据变成空值

```

[3]:
import pandas as pd
pd.to_*?

pd.to_datetime
pd.to_numeric
pd.to_pickle
pd.to_timedelta

```

通配符解决tab 键代码提示禁用的问题

LabelEncoder() sklearn中的数据标准化方法

- [红, 黄, 蓝] 转换成[1, 2, 3]
- pd.get_dummies()
 - 将列中的所有字段的值都转换成一个数值列

```
# 选择相关特征进行建模
X = df[['Your gender', 'How important is exercise to you ?', 'How healthy do you cons:
X = pd.get_dummies(X) # 将分类变量转为数值变量

# 将年龄段转为数值
y = _____

# 将数据集划分为训练集和测试集
X_train, X_test = _____

# 创建随机森林模型
rf_model = _____
```

• 机器学习类比人的学习过程

- 小名学习步骤
 - 报班找老师- 什么训练模型，学习训练多少次
 - 小明拿着作业和答案开始学习
 - 小明将知识都记录在笔记本上面，下次还可以用
 - 小明的班级开始考试，给了小明一份试卷
 - 小明将y_pred 做完后交卷，完整试卷包含小明答案的结果results.txt
 - 老师批改试卷，给小明一个成绩单
 - 小明妈妈说重新学习，对第一次学习的结果不满意
 - 重新学习的方法
 - 第二次考试

• 评估报告

- 手写方法: 2.1.1 (y_test==y_pred).mean()
- model.score方法
 - 看备注，如果指明训练集，测试集
 - model.score(X_train, y_train)
- sklearn.metrics 方法
 - 误差方法(标准答案，小明的结果)

```
results_df.to_csv('2.2.3_results.txt', index=False)
# 使用测试工具对模型进行测试，并记录测试结果
train_score = rf_model.score(X_train, y_train) # 训练集分数
test_score = rf_model.score(X_test, y_test) # 测试集分数
mse = mean_squared_error(y_test, y_pred) # 均方误差
r2 = r2_score(y_test, y_pred) # 决定系数

with open('2.2.3_report.txt', 'w') as report_file:
    report_file.write(f'训练集得分: {train_score}\n')
    report_file.write(f'测试集得分: {test_score}\n')
    report_file.write(f'均方误差(MSE): {mse}\n')
    report_file.write(f'决定系数(R^2): {r2}\n')
```